

Opakovací cvičení: Python Lekce N01 až N20

Tento úkol prověří tvé znalosti pokročilejších konstrukcí v jazyce Python. Zaměříme se na objektově orientované programování (OOP), práci s výjimkami, dekorátory, správu kontextu (Context Managers) a základy asynchronního programování. Můžeš použít AI integrované v VSCODE (žádné externí webové AI, jako je chatGPT, apod.).

Příprava

Otevři si připravený soubor se šablonou: `n20_python_opakovani_SABLONA.py`. V tomto souboru najdeš předpřipravený kód malého skladového systému, kde na několika místech chybí implementace. Tvé úkoly jsou vyznačeny jako **ÚKOL 1** až **ÚKOL 5**.

Zadání úkolů

ÚKOL 1: Ošetření skladu (Lekce N05, N08)

Ve třídě `Zbozi` je připravena metoda `odeber(self, pocet: int)`. Tvým úkolem je tuto metodu implementovat.

- Zkontroluj, zda požadovaný `pocet` na odebrání není větší než aktuální množství zboží na skladě (privátní atribut `self.__mnozstvi`).
- Pokud je požadavek větší než skladová zásoba, **vyhod' výjimku** `NedostatekZboziError` (tato výjimka je již v kódu definována) se smysluplnou textovou zprávou (např. `"Na skladě není dostatek kusů!"`).
- V opačném případě hodnotu `self.__mnozstvi` o zadaný parametr `pocet` sniž.

ÚKOL 2: Hezký uživatelský výpis pomocí Dunder metod (Lekce N10, N13)

Doplň implementaci magické (dunder) metody `__str__` ve třídě `Zbozi`, která se volá například při slovech `print(zbozi)`.

- Pomocí **f-stringu** vrať uživatelsky přívětivý řetězec obsahující název položky, její cenu a počet kusů skladem.
- **Formát by měl vypadat takto:** `"Zboží Kniha Python | 600.00 Kč | 10 ks skladem"`
- Cenu (z důvodu haléřů) vhodně naformátuj ve f-stringu na dvě desetinná místa (pomocí znaku specifikátoru `:.2f`).

ÚKOL 3: Bezpečný zápis dat pomocí Context Managera (Lekce N07, N18)

Ve třídě `ZapisovacDatabase` musíš dopsat chování ukončovací metody `__exit__(self, exc_type, exc_val, exc_tb)`. Tato třída se používá ve `with` bloku jako Context Manager k ukládání dočasných záznamů (`self.data`) do JSON souboru (jméno je v `self.soubor`).

- V metodě `__exit__` zkontroluj, zda v průběhu využívání bloku nedošlo k havárii (tzv. `exc_type` je prázdné/`None`).

- **Pokud nedošlo k chybě:** Zapišeš logy (`self.data`) reálně přes modul JSON na pevný disk (nápopěda: použij `with open(...)` klasicky uvnitř metody a k zápisu JSON funkce `json.dump`).
- **Pokud došlo k chybě:** Místo zapisování vypiš do konzole varování o zrušení (rollbacku) transakce z důvodu havárie a data nijak a nikam **neukládej**.

ÚKOL 4: Filtrování dat a List Comprehension (Lekce N03, N15)

Doplň implementaci ve spouštěcí funkci `filtruj_a_zlevni(zbozi_list, max_cena, sleva_procenta)`.

- Funkce přijímá velké pole instance zboží. Tím proiteruj (ideálně přes jeden průhledný řádek s pomocí list comprehension).
- Vyber pouze a jen to `Zbozi`, jehož originální cena je roven, a nebo menší hodnotě z parametru `max_cena`.
- Všem takto "přeživším" produktům pak aktivně a trvale uprav (zkrat) jejich nastavenou cenu přímo na instanci o to zadané procento (`sleva_procenta`). Příklad: 20% sleva znamená, že po výpočtu má produkt novou cenu jen 80% originální sumy.
- Tento nový vyfiltrovaný (a plošně zlevněný) list položek nakonec metoda standardně vrátí.

ÚKOL 5: Asynchronní smyčka a Event Loop (Lekce N20)

Utečme na chvíli od OOP; v dolní sekci je prázdná tzv. korutina (asynchronní funkce začínající `async def` názvem `over_dostupnost_api()`).

- Tato metoda hraje simulaci asynchronního (časově zpožděného) dotazování síťového centrálního API skladu dodavatelů.
- Tvým úkolem je donutit funkci, ať se uvnitř nejprve na přesně plnou **jednu vteřinu zastaví**. Využij standardní asynchronní prodlení z knihovny `asyncio` s operátorem `await` (nesmíš použít krutý zasekávací a zakázaný `time.sleep`, to by spící task nesrovnal do fronty událostí).
- Po zdárném probuzení o jedno-vteřinové chvílce nezbyvá než simulativně přes `return` vrátit hardcode potvzovací hodnotu `True`.

Odevzdání a testování šablony

Jakmile naplníš všechny slepé `pass` uličky a chybějící prvky napříč funkcemi ve tvé šabloně kódů, zavolej celý skript z prostředí příkazové řádky či terminálu vývojového prostředí:

```
python n20_python_opakovani_JMENO_PRIJMENI.py
```

Pokud tě nikdo neseřve errorovou hláškou, mělo by fungovat následující (v souladu s dole definovaným blokem `def main()`):

1. V terminálu bez chyby proběhnou hezké tisky Zbozi z 2. úkolu. Zlevněné zboží by mělo zobrazovat nové ponížené ceny (jako 320 Kč pro levnou Myš za plných 400,- Kč a podobně).
2. Tisk ti vypravuje varovnou hlášku: "BOMBA! Výjimka úspěšně zachycena programem: Na skladě není dostatek kusů". Úkaz funkčnosti Úkolu č. 1 s házením vlastního Erroru. A tím pádem také otestování Úkolu č. 3, neb soubor `sklad_export.json` na disku NIKDY fyzicky nevznikne vinou ochrany před touto spuštěnou krumpáčovou havárií.

3. Posledním testem na tvé síťové znalosti budou vytištěné řádky: --- Start asynchronní prověrky ..., za kterým skript opravdu prokazatelně udělá pouhou jednovteřinovou (za to paralelní dýchatelnou) zastávku a suverenně úhledně dotiskne ... --- Asynchronní prověrka dokončena ---.